

---

## Oliver Hauke

ist Mathematiker und IT-Referent im Kompetenzzentrum „Analyse und Auswertung“ des Statistischen Bundesamtes. Er verantwortet die Weiterentwicklung der technischen Infrastruktur des Forschungsdatenzentrums und berät das Netzwerk empirische Steuerforschung in der effizienten Nutzung der Systeme.

## Annette Kristiansen

ist Ökonomin und Referentin im Referat „Unternehmenssteuern“ des Statistischen Bundesamtes. Sie beschäftigt sich mit der Zusammenführung der Unternehmenssteuerstatistiken und betreut das Netzwerk empirische Steuerforschung. Für ihre externe Promotion an der Universität Bremen untersucht sie die technologische Vielfalt multinationaler Unternehmen.

# EFFIZIENTE ANALYSE GROSSER FORSCHUNGSDATEN MITHILFE VON R

---

Oliver Hauke, Annette Kristiansen

---

🔑 **Schlüsselwörter:** Effiziente Analysen, R-Programmierung, Forschungsdaten, Unternehmenssteuerstatistik, Business-Tax-Panel, Netzwerk empirische Steuerforschung

## ZUSAMMENFASSUNG

Das Forschungsdatenzentrum des Statistischen Bundesamtes baut seine technische Infrastruktur gezielt aus, um der Wissenschaft erweiterte Analysemöglichkeiten amtlicher Daten bereitzustellen. Seit November 2025 steht Forschenden exklusiver Zugang zu diesen erweiterten Systemressourcen in R und Stata zur Verfügung. R-basierte Tools – insbesondere Apache Arrow, Parquet und DuckDB – ermöglichen es, gängige Analysen großer Forschungsdatensätze in Echtzeit auszuführen. Am Beispiel des Business-Tax-Panel (2013 bis 2019) werden die erzielbaren Effizienzgewinne bei der Auswertung umfangreicher steuerstatistischer Daten im Rahmen des Netzwerks empirische Steuerforschung demonstriert.

🔑 **Keywords:** *Efficient analysis – R programming – research data – corporate tax statistics – Business Tax Panel – empirical tax research network*

## ABSTRACT

*The Research Data Centre at the Federal Statistical Office is strategically expanding its technical infrastructure to provide researchers with enhanced analytical capabilities for official data. Since November 2025, researchers have had exclusive access to these extended system resources in R and Stata. R-based tools – particularly Apache Arrow, Parquet, and DuckDB – enable real-time analysis of large research datasets. Using the Business Tax Panel (2013–2019), this paper demonstrates the achievable efficiency gains in analysing large-scale tax statistics within the empirical tax research network.*

## Inhaltsverzeichnis

---

|               |                                                                |           |
|---------------|----------------------------------------------------------------|-----------|
| 3             | Datenverarbeitungsmethoden . . . . .                           | 3         |
| 3.1           | Ausgangslage . . . . .                                         | 3         |
| 3.2           | Anforderungen an Datenverarbeitungsmethoden . . . . .          | 3         |
| 3.3           | Geeignete Datenverarbeitungsmethoden in R . . . . .            | 3         |
| 4             | Vergleich der Performanz . . . . .                             | 4         |
| 4.1           | Benchmark-Setup . . . . .                                      | 4         |
| 4.2           | Baselines in SAS und Stata . . . . .                           | 5         |
| 4.3           | Kleiner Vergleich zwischen R und den Baselines . . . . .       | 5         |
| 4.4           | Größere Datenmengen in R . . . . .                             | 7         |
| 4.5           | Erkenntnisse des Experiments . . . . .                         | 7         |
| 6             | Fazit . . . . .                                                | 8         |
| 6.1           | Kernaussage des Artikels . . . . .                             | 9         |
| 6.2           | Auswirkung auf IT-Infrastruktur und Beratung . . . . .         | 9         |
| 6.3           | Auswirkungen auf die Statistikproduktion . . . . .             | 9         |
| 6.4           | Auswirkungen auf die FDZ . . . . .                             | 9         |
| 6.5           | Auswirkung auf die Wissenschaft . . . . .                      | 9         |
| 6.6           | Schlussbemerkung . . . . .                                     | 9         |
| <b>Anhang</b> |                                                                | <b>10</b> |
| 7             | Weitere Performancemessungen für kleinere Datenmengen. . . . . | 10        |

## 3

## Datenverarbeitungsmethoden

### 3.1 Ausgangslage

Wir betrachten in dieser Arbeit die Performanz sog. Datenverarbeitungsmethoden, d.h. Softwarelösungen, die Daten einlesen, modifizieren, aggregieren, auswerten oder analysieren. Das Schreiben von Daten wird in unserer Untersuchung ausgeklammert, da es sich bei den Veröffentlichungen des Statistischen Bundesamtes oder den Outputs der Wissenschaft im FDZ um kleine Tabellen handelt, die keine große Performanz erfordern. Dennoch bieten die hier vorstellten Lösungen auch hohe Performanz im Schreiben von Daten.<sup>1</sup>

Daneben wächst der Einsatz von R stetig im StBA und FDZ stetig.

Empfehlung des FDSZ der deutschen Bundesbank Gommelka, Blaschke und Hirsch (2021) ....

### 3.2 Anforderungen an Datenverarbeitungsmethoden

- › Performanz und Nutzererfahrung auf dem aktuellen Stand der Technik
- › leicht einzurichten auch in den abgeschotteten System der amt. Statistik (tlw. langwierig und schwierig), d.h. wenig Abhängigkeiten und unabhängig von Internet bzw. Cloud betreibbar.
- › langfristig stabil und gut unterstützt
- › leicht nutzbar, gut dokumentiert, gutes Schulungsangebot

### 3.3 Geeignete Datenverarbeitungsmethoden in R

- › R ist die native Programmiersprache von Statistikern/-innen und Daten-affinen Programmierer/-innen. Frei verfügbar und leicht einzurichten. Somit weit verbreitet, insb. an Universitäten.

- › R ist Open Source und wird ständig weiterentwickelt (aktuell in Version 4.5 verwendet, jährliche Major Releases).
- › Eine weltweit aktive Community löst Probleme zeitnah und ergänzt stetig Funktionalitäten in Form von sog. *R-Packages*.
- › Bietet nativ bereits viele Datenverarbeitungsmethoden, die jedoch durch diese R-Packages ganz neue Level an Nutzerfreundlichkeit, Funktionsumfang und Performanz erreichen.

#### 3.3.1 Klassische Methoden. Base R.

**Vorteile:** Einfach, weit verbreitet, keine zusätzlichen Abhängigkeiten **Nachteile:** Langsam bei großen Daten, hoher Speicherverbrauch, alle Daten müssen in RAM passen

**Empfehlung:** Nur für kleine Datensätze (<1 GB) oder wenn Kompatibilität wichtiger als Performance ist

- › base-r Grundlage/einstieg, weder sonderlich anschaulich noch sonderlich performant -> Packages überlassen

Zweckdienlich für Einsteiger oder gelegentliches Nutzer oder falls gar keine Abhängigkeiten in Form von Pkg erlaubt sind. Ansonsten...

#### data.table.

**Vorteile:** Sehr schnell für In-Memory-Operationen, ausgereift, kompakte Syntax **Nachteile:** Alle Daten müssen in RAM passen, steile Lernkurve, keine Out-of-Core-Unterstützung

**Empfehlung:** Gute Alternative wenn Datensatz in RAM passt und keine Parquet-Integration benötigt wird

-> {data.table} in den meisten Fällen die performanteste klassische DVM. An die Syntax an base-r angesiedelt und nur hier verwendet, spezielles Wissen notwendig.

- › {tidyverse} Ökosystem von Packages zur Datenverarbeitung. bekannt für besonders hohe Nutzerfreundlichkeit und einer eigenen Programmiersyntax sog. {tidy}-Syntax, die den Anspruch hat menschlicher Sprache abzubilden:

[ Tabelle Beispiel Dplyr ]

#### dplyr + dbplyr.

**Vorteile:** Vertraute dplyr-Syntax, gut mit Datenbanken integriert **Nachteile:** Langsamer als Arrow/DuckDB, benötigt Backend-Datenbank, SQL-Translation manchmal limitiert

**Empfehlung:** Wenn bereits PostgreSQL- oder ähnliche Datenbank-Infrastruktur vorhanden ist

<sup>1</sup> [www.opencode.de/URL-To-Be-Announced](http://www.opencode.de/URL-To-Be-Announced).

So beliebt, dass die Community sie vielen anderen Packages ergänzt hat. Bspw. für `{data.table}` die Packages `{dtplyr}` oder `{tidytable}`. Wie im Falle von `{dtplyr}` kann es dabei aber zu Einschränkung der Performanz kommen.<sup>12</sup>

**3.3.2 Ausgewählte Best Practices.** in Abschnitt Kapitel 3 beschriebenen Methoden (Variablenselektion, Datenaufteilung):

**3.3.3 Hoch-performante R-Packages.** Definition einfach: können nativ Daten über RAM verarbeiten.

von Desktopanwendung unterm Tisch zu Performanten Analyseserver paradigmwechsel notwendig: in RAM vs nicht im RAM!

Die auf den Servern des StBA und FDZ verfügbaren Packageversionen lassen sich aus organisatorischen Gründen nicht flexibel anpassen (kein Internet, kuration intern, s. A. Inf) können Tabelle X entnommen werden.

Wie in Abschnitt ?@sec-infrastruktur beschrieben war es uns aus organisatorischen Gründen nicht möglich, die neuste Version aller Pakete zu verwenden. Die leichter nutzbaren Variationen `{duckplyr}` bzw. `{tidytable}` zu nutzen. Die Package wurden aber wenige Tage nach dem Experiment eingerichtet. Die Syntaxtranslation kann dabei die Performanz verschlechtern, sodass bei `{duckdb}` bzw. `{data.table}` zwischen Performanz und Code-Verständlichkeit abgewägt werden muss. `{arrow}` ist schon länger als `{polars}` im Statistischen Bundesamt verfügbar, sodass wir darin bereits mehr Erfahrungen einsetzen konnten.

Die Analyse großer Datensätze wie des in der FDZ-Umgebung stellt besondere Anforderungen an die verwendeten Werkzeuge. Traditionelle Ansätze (CSV-Dateien, Base R) stoßen bei dieser Datengröße an ihre Grenzen. Moderne spaltenorientierte Technologien bieten hier deutliche Vorteile.

beachte/erwähne: <https://www.r-bloggers.com/2024/04/the-truth-about-tidy-wrappers-3/>

## 4

### Vergleich der Performanz

<sup>12</sup>Lediglich in der Arbeitsspeicherbeanspruchung, ist SAS deutlich besser als klassische R-Methoden. Gegenüber hoch-performanten R-Methoden benötigt SAS jedoch den 10-fachen Arbeitsspeicher.

### 4.1 Benchmark-Setup

Um zwischen den in Abschnitt Kapitel 3 vorgestellten Datenverarbeitungsmethoden abzuwägen, haben wir deren Performanz in einem systematischen Benchmark-Experiment auf dem SAS-Server des Statistischen Bundesamtes und den neuen OnSite R- und Stata-Servern des FDZ gemessen. Weiterführende Benchmarks sowie der vollständig reproduzierbare Code sind auf [Opencode](https://www.opencode.de/URL-To-Be-Announced) verfügbar<sup>13</sup>.

Für die Untersuchung wurden simulierte Einzeldaten in verschiedenen Größen (0,1%/1%/10% von den insgesamt 80 Mio. Beobachtungen des BTP) betrachtet. Sie wurden aus öffentlich verfügbaren Informationen generiert, um die wesentlichen Strukturen des BTP für unsere Analyse künstlich nachzubilden. Abgrenzend zu den im FDZ bereitgestellten *Datenstrukturfiles* wurden die Testdaten nicht aus echten Mikrodaten abgeleitet. Der simulierte Datensatz ist ausschließlich für technische Testzwecke der nachfolgenden Anwendungsfälle bestimmt:

1. **Fallzahlen** (pro Statistik und Jahr),
2. **Regression** (Einflussfaktoren auf Verlustvorträge).

Um eine zuverlässige Performanz-Messung zu erhalten, wurde jede Datenverarbeitungsmethode dreifach angewandt. Der Vergleich erfolgt anhand der mittleren Messung der folgenden Zielgrößen:

1. **Arbeitsspeicher** der Auswertung,
2. **Laufzeit** der Auswertung (real verstrichene Zeit),
3. **Festplattenspeicher** der verwendeten Daten.

Die Zielgrößen sind in dieser Reihenfolge zu priorisieren, da ein übermäßiger Arbeitsspeicherverbrauch zu schwerwiegenden Abbrüchen führen kann, während hohe Laufzeiten dennoch zu einem erfolgreichen Ergebnis führen können. Festplattenspeicher ist ausreichend vorhanden, aber unnötige Belegung sollte vermieden werden.

Feine Abstufungen der Zielgrößen sind für unsere Aufgaben nicht gleichermaßen relevant. Eine Methode, die die betrachteten Auswertungen in unter 10 Sekunden mit weniger als 4 GB Arbeitsspeicher durchführen kann, wird bereits als optimal bewertet. Weitere Optimierungen sind in diesem Fall nicht erforderlich. Falls bereits die gewünschte Performanz erreicht wurde, rückt die Reduktion des Entwicklungsaufwand in den Vordergrund. Entsprechend sind Aspekte wie einfache Anwendbarkeit und Nachvollziehbarkeit – selbst für unerfahrene Nutzende – ausschlaggebend.

<sup>13</sup>[www.opencode.de/URL-To-Be-Announced](https://www.opencode.de/URL-To-Be-Announced).

Tabelle 1

Versionen betrachteter R-Pakete

|            | [t]      |
|------------|----------|
| data.table | 1.17.8   |
| tidyverse  | tba      |
| vroom      | tba      |
| arrow      | 20.0.0.2 |
| duckdb     | 1.3.2    |
| polars     | 1.0.1    |

4.2 Baselines in SAS und Stata

Klassisch werden die betrachteten Auswertungsaufgaben im Statischen Bundesamt hauptsächlich mit SAS und im FDZ mit Stata gelöst. Um die nachfolgenden Ergebnisse einzuordnen, haben wir die Performanzmessungen für 1% des simulierten BTP-Daten (etwa. 800.000 Beobachtungen) in SAS und Stata durchgeführt.

Die Ergebnisse in Tabelle 2 zeigen, dass der umfangreiche Arbeitsspeicher des FDZ OnSite-Servers nicht ausreicht, um das ganze BTP in Stata einzulesen, sodass zwingend mit Variablenselektion gearbeitet werden muss. SAS zeigt seine Stärke anhand der geringen Beanspruchung des Arbeitsspeichers, benötigt aber mit etwa 2,3 Minuten pro Auswertung die doppelte Laufzeit gegenüber Stata.

> und nur mit **5 GB Festplattenspeicher** (6-fach weniger gegenüber SAS und Stata).

Unter den klassischen R-Methoden bietet {data.table} die beste Performanz. Dabei ist wesentlich zu beachten, dass die Einlesefunktion `fread` direkt die relevanten Variablen auswählt. Dadurch lassen sich unsere Auswertungen um einen Faktor 3 beschleunigen und vor allem der Arbeitsspeicher um einen Faktor 1.000 reduzieren.

Alle hoch-performanten Methoden sind dermaßen effizient, dass die Datenmenge von 800.000 Beobachtungen nicht mehr ausreicht, um zwischen den besten Methoden zu differenzieren. Daher erhöhen wir im nächsten Experiment die Datenmenge auf 8.000.000 Beobachtungen (10% BTP).

4.3 Kleiner Vergleich zwischen R und den Baselines

Für 1% der simulierten BTP Daten, reduzieren die R-Datenverarbeitungsmethode alle Zielgrößen gegenüber SAS und Stata erheblich (siehe Tabelle 3 und Tabelle 4). Selbst klassische R-Methoden sind um den Faktor 7 schneller und benötigen nur 33 % des Festplattenspeichers. {polars}, {duckdb} und {arrow} lösen die betrachteten Auswertungen sogar

- > in **unter 2 Sekunden** (27-fach schneller),
- > mit unter **300 MB Arbeitsspeicher** – {duckdb} und {polars} sogar mit unter **5 MB** (10-fach weniger)<sup>|4|5</sup>

<sup>|4|</sup>Lediglich in der Arbeitsspeicherbeanspruchung, ist SAS deutlich besser als klassische R-Methoden. Gegenüber hoch-performanten R-Methoden benötigt SAS jedoch den 10-fachen Arbeitsspeicher.

<sup>|5|</sup>{arrow} benötigt mit 274 MB deutlich mehr Arbeitsspeicher als {polars} und {duckdb}, aber kommt dennoch mit minimalen Entwicklungsumgebungen aus, bspw. einem standardmäßigen Laptop.

Tabelle 2

Baselines in SAS und Stata (anhand 1% des simuliertes BTP)

|       |            | [t]   |           |       |
|-------|------------|-------|-----------|-------|
| SAS   | Fallzahl   | 65 MB | 2.4 Min.  | 33 GB |
| SAS   | Regression | 7 MB  | 2.8 Min.  | 33 GB |
| Stata | Fallzahl   | 33 GB | 55,2 Sek. | 32 GB |
|       | Regression | 33 GB | 55,8 Sek. | 32 GB |

Tabelle 3

Performanz Messungen für die Fallzahlberechnung anhand 1% des BTP

|              |         | [t]                            |          |          |         |
|--------------|---------|--------------------------------|----------|----------|---------|
| {polars}     | Parquet | -                              | 3,3 MB   | 1,0 Sek  | 4,9 GB  |
| {duckdb}     | Parquet | -                              | 4,8 MB   | 1,5 Sek  | 4,9 GB  |
| {arrow}      | Parquet | Variablenauswahl               | 273,7 MB | 1,7 Sek  | 4,9 GB  |
| {data.table} | CSV     | Variablenauswahl               | 42,9 MB  | 2,5 Sek  | 13,0 GB |
| {arrow}      | Parquet | Datenaufteilung (gleichmäßig)  | 229,7 MB | 2,8 Sek  | 4,9 GB  |
| {arrow}      | Parquet | -                              | 272,8 MB | 2,8 Sek  | 4,9 GB  |
| {arrow}      | Parquet | -                              | 230,4 MB | 3,8 Sek  | 4,9 GB  |
| {arrow}      | Parquet | Datenaufteilung (Berichtsjahr) | 229,9 MB | 3,9 Sek  | 4,8 GB  |
| {data.table} | CSV     | -                              | 33,7 GB  | 7,4 Sek  | 13,0 GB |
| {data.table} | CSV     | Datenaufteilung (Berichtsjahr) | 50,4 GB  | 13,6 Sek | 10,5 GB |

Tabelle 4

Performanz Messungen für die Regressionsberechnung anhand 1% des BTP

|         |         | [t]                            |          |          |         |
|---------|---------|--------------------------------|----------|----------|---------|
| {arrow} | Parquet | Datenaufteilung (Berichtsjahr) | 529,7 MB | 3,8 Sek  | 4,8 GB  |
| {dplyr} | CSV     | Variablenauswahl               | 154,8 MB | 13,2 Sek | 13,0 GB |

## 4.4 Größere Datenmengen in R

Auch für 10 % der simulierten BTP-Daten, kann R die beiden betrachteten Auswertungen in Echtzeit mit minimalem Arbeitsspeicher beantworten (siehe Tabelle 5 und Tabelle 6). Die Auswertungsmethoden mit der höchsten Performanz nutzen alle das {parquet} und erreichen:

1. {arrow} benötigt nur **4,7 Sekunden** und 230 MB Arbeitsspeicher,
2. {duckdb} benötigt nur 5,1 Sekunden und **4,7 MB Arbeitsspeicher**.

Unerwarteterweise ist die Laufzeit von {polars} für diese Datenmenge mit 1,2 Minuten weit abgeschlagen, obwohl das gleiche Auswertungsskript für kleinere Datenmengen immer die geringste Laufzeit aufwies. Da {polars} das jüngste der betrachteten R-Packages ist, vermuten wir Performanzprobleme in der vorliegenden Version 1.0.1<sup>16</sup>. Zudem hat es noch größere Entwicklungsaufwände beansprucht, da der an Python angelehnte Syntaxstil deutlich von üblicher Programmierung in R abweicht. Öffentliche Hilfestellung bezieht sich hauptsächlich auf Python (oder Rust), was sich in den Antworten der KI-Chatassistenten widerspiegelt.

Klassische Methoden kommen bei der betrachteten Datenmenge bereits an ihre Grenzen. Falls in {data.table} unbedacht der vollständige Datensatz eingelesen wird, werden in unserem Anwendungsfall die verfügbaren 512 GB Arbeitsspeicher vollständig beansprucht. Wir klassische Methoden ist somit deutlich mehr Erfahrung der Nutzenden notwendig, um die relevanten Variablen manuell geschickt zu selektieren unter Beobachtung des Arbeitsspeicherverbrauchs.

Insgesamt kommen wir zu dem Ergebnis, dass {arrow} kombiniert mit {parquet} im Gesamtpaket die beste Datenverarbeitungsmethode für die Bewältigung unserer Aufgaben aus den folgenden Gründen ist:

1. Für die relevanteste Datenmenge zeigte {arrow} die geringsten Laufzeiten unter 5 Sekunden, was der Fachstatistik Antworten auf Sonderanfragen in Echtzeit ermöglicht und der Wissenschaft alle denkbaren explorativen Analysen.
2. auch die Nutzererfahrung war mit {arrow} am besten und die damit einhergehenden Entwicklungsaufwände waren dadurch am geringsten.
3. die anschauliche {tidyverse} Syntax lässt sich zum Großteil änderungsfrei und ohne Performanzverluste auf {arrow} übertragen. {duckdb} kann bei

Kenntnissen in SQL zu bevorzugen sein und {polars} mit Erfahrungen in Python.

4. Beide Anwendungen ließen sich in unter 20 nachvollziehbaren Zeilen in {arrow} programmieren.
  5. Obwohl nicht alle Features aus {tidyverse} für {arrow} verfügbar sind, haben wir bisher immer eine Lösung gefunden, um die gewünschte Auswertung umzusetzen, bspw. durch Kombination mit {duckdb}.
  6. {arrow} spart ausreichend Arbeitsspeicher (mit Verbrauch unter 1 GB) (tba: Fußnote zu duckdb/polars). Der Arbeitsspeicherverbrauch von {arrow} ist in unserer Auswertung nahezu konstant abhängig von der Datenmenge, sodass {arrow} leicht auf noch größere Datenmengen skalieren kann
  7. Dadurch das {arrow} die Nutzung auch von mehreren {parquet} Dateien unterstützt spart gegenüber CSV mind. 50% der Festplatte (für das reale BTP sogar 80%+) und ermöglicht schnelle Nutzung auch von Datenmengen überhalb des verfügbaren Arbeitsspeichers.
- › {arrow} zeigte im Experiment die beste Nutzendenerfahrung.
- › Beide Anwendungen lassen sich in unter 20 Zeilen mit Arrow programmieren (verständlich lesbar) s. Opencode Repo.
  - › Leicht und verständlich - adaptiert nativ schöne {tidyverse} Syntax. Dagegen sind {duckdb} und {polars} auf Zusatzpakete angewiesen, die tlw. die Performanz reduzieren können.
  - › Flexibel, selektives Einlesen ist zwingend erforderlich
  - › hohe Stabilität, fehlende Features können durch leicht in duckdb oder in den Arbeitsspeicher überführt und dennoch ausgeführt werden
- › kleiner Nachteile: Erfahrung notwendig, da...
- › erst Datenaufteilung/selektives Einlesen die maximale Performanz erreicht
  - › aktuell nicht der volle Funktionsumfang von {tidyverse} in {arrow} abgebildet wird, sodass auf fehlende Features mit {duckdb} etc. reagiert werden muss.

## 4.5 Erkenntnisse des Experiments

- › Auswahl der optimalen hoch-performanten Methode erfordert eine Untersuchung in der eigenen Infrastruktur anhand eigener Daten. Jedoch bieten alle HP-Methoden für die meisten Anwendungsfälle starke Performanzgewinne und sollten eingesetzt werden (nach den vorhandenen Erfahrungen und Strukturen: tidy

<sup>16</sup>Das R-Package {polars} Version 1.0.1 war nur der erste Minor Release nach einer kompletten Überarbeitung. Inzwischen sind 5 neuere Major Releases mit diversen Performanz-Verbesserungen verfügbar.



Tabelle 5

Performanz Messungen für die Fallzahlberechnung anhand 10% des BTP

|              |         | [t]                            |          |          |          |
|--------------|---------|--------------------------------|----------|----------|----------|
| {arrow}      | Parquet | Datenaufteilung (gleichmäßig)  | 229,9 MB | 4,7 Sek  | 48,8 GB  |
| {duckdb}     | Parquet | -                              | 4,7 MB   | 5,1 Sek  | 49,4 GB  |
| {arrow}      | Parquet | Variablenauswahl               | 702,1 MB | 7,5 Sek  | 49,4 GB  |
| {arrow}      | Parquet | Datenaufteilung (Berichtsjahr) | 4,7 MB   | 11,7 Sek | 48,1 GB  |
| {arrow}      | Parquet | -                              | 230,4 MB | 17,0 Sek | 49,4 GB  |
| {arrow}      | Parquet | -                              | 701,2 MB | 20,6 Sek | 49,4 GB  |
| {data.table} | CSV     | Variablenauswahl               | 427,5 MB | 23,6 Sek | 130,2 GB |
| {polars}     | Parquet | -                              | 3,3 MB   | 1,2 Min  | 49,4 GB  |
| {data.table} | CSV     | -                              | 336,0 GB | 2,2 Min  | 130,2 GB |
| {data.table} | CSV     | Datenaufteilung (Berichtsjahr) | 506,4 GB | 5,0 Min  | 104,6 GB |

Tabelle 6

Performanz Messungen für die Regressionsberechnung anhand 10% des BTP

|         |         | [t]                            |          |         |          |
|---------|---------|--------------------------------|----------|---------|----------|
| {arrow} | Parquet | Datenaufteilung (Berichtsjahr) | 782,1 MB | 5,2 Sek | 48,1 GB  |
| {dplyr} | CSV     | Variablenauswahl               | 1,5 GB   | 1,9 Min | 130,2 GB |

vs Python vs SQL). Falls die Daten in CSV unter 25% des Arbeitsspeichers sind, sind auch klassische Methoden sehr gut nutzbar (bevorzugt {data.table}).

› **Package für optimale Laufzeit ist nicht eindeutig.** Je nach Datenmenge hat sich die Top 3 stark unterschieden.

- › 1e6: arrow > duckdb > DT+SEL > polars
- › 1e5: polars > duckdb > arrow > DT+SEL
- › 1e4: polars > data.table > arrow > readr

› Unerwarteterweise ist {polars} bis 1% BTP am schnellsten aber für das 10% BTP abgeschlagen (unter den schlechtesten Methoden).

› Dagegen ist **das effizienteste betrachtete Datenformat über 1000 Beobachtungen eindeutig Parquet** – sowohl in Speicherbedarf als auch in Lese- und Schreibzeiten.

› HP-Methoden und Optimierungen können erst ab einer Mindestdatenmenge vorteilhaft sein. Insb. Datenaufteilung bietet nur HP Methoden ab einer gewissen Datenmenge Vorteile. Für kleine Mengen überwiegt der Overhead durch das Lesen mehrerer kleiner Dateien. Klassische Methoden werden durch eine Datenaufteilung nur negativ beeinflusst.

› **Variablenauswahl beim Einlesen ist immer vorteilhaft.** High-Performance Methoden können aber bei einer **Datenaufteilung** darauf verzichten.

› (tba: mit obigem zusammenführen) Unter den klassischen Methoden ist {data.table} mit Variablenauswahl am effizientesten und schneller als eine schlechte Anwendung von {arrow}. D.h. ein {data.table} Profi, ist besser als ein schlechter Arrow Programmierer, der aber größeres Potenzial für Verbesserungen hat.

Anfängern ist eher {arrow} zu empfehlen, da in {data.table} die Variablenauswahl abhängig von dem verfügbaren Arbeitsspeicher manuell gesteuert werden muss und daher nicht so flexibel ist.

- › 5x langsamer als {arrow}
- › Variablenauswahl ab einer gewissen Datenmenge erforderlich, da ohne 3x größer als die Eingangsdaten, dauert 2.2 min anstatt 23 Sec. -> schwierige Anwendung (Entscheidung relevanter Variablen muss vorab getroffen werden unter Beachtung der Arbeitsspeicher Restriktionen)
- › Datenaufteilung verschlechtert DT im Gegensatz zu arrow

- › {arrow} verzeiht Fehler leicht(er als DT). Denn schlecht programmiert, ist es immernoch ganz gut). Alleine durch die Datenaufteilung (vorgegeben durch die Datenbereitstellung, vorbereitet durch das FDZ), erreichen wir die Fallzahl-Berechnung in 11.73s, nur 5s über dem absoluten Optimum.
- › R-base oder readr sind immer weit abgeschlagen und zu vermeiden.
- › Wenn der RAM extrem begrenzt ist, können {polars} oder {duckdb} dennoch eingesetzt werden.

## 6

## Fazit



## 6.1 Kernaussage des Artikels

Wir empfehlen eine Datenhaltung im Parquet-Format und **R** incl. {arrow} + {parquet} (bei Bedarf {duckdb}, {polars}) zu erproben und zu etablieren! Denn unsere Untersuchung demonstriert ganz neue Möglichkeiten, analytische Fragestellungen anhand großer Datenmengen in Echtzeit zu beantworten.

## 6.2 Auswirkung auf IT-Infrastruktur und Beratung

- › Fokus auf Hardware und Software mit größtem Mehrwert (weniger RAM und R anstatt Spark, GPUs, Datenbanken, Cloud, ...)
- › Fokus auf Maintainance und Patching von R, Parquet und hier genannte Packages {arrow}, {duckdb}, {polars}
- › Etablierung dieser Softwarelösungen kann langfristig...
  - › die Hardwareanforderungen reduzieren
  - › die Analysesoftwarevielfalt verschlanken, somit Beschaffungen einsparen (besserer Ausdruck?)
- › Schulungsangebot benötigt für *“hoch-performante Auswertungen in R”*
- › eigener Beitrag zu Open Source entscheidend, um die Etablierung zu fördern.

## 6.3 Auswirkungen auf die Statistikproduktion

- › Steigerung der Effizienz und Reaktionsgeschwindigkeit
- › (Vorsichtig!?! Annette, lieber weg oder rein?) Vereinfachung der Statistikproduktion, durch leicht nachvollziehbaren Code und Fokus auf Wissensaufbau in R
- › Aufbau eines Datenbestands in Parquet
- › Anwendung auf weitere Produkte
  - › Überarbeitung des BTP
  - › Mindeststeuer (oder?)
  - › DRG
  - › TPP 2

## 6.4 Auswirkungen auf die FDZ

- › Berücksichtigung in Mustersyntaxen und Nutzendenberatung in FDZ Bund. Teilen von Code-Snippets und Templates fördern.
- › mögliche Vereinfachung der Betreuung (Fokus auf R, Reduktion von Performanz- und Programmierproblemen)

- › Bereitstellung von Parquet in geeigneter Aufteilung empfohlen
- › Erprobung im FDZ Bund Vorbild für andere FDZ
- › Optimierung von Hardware und Software Angebot

## 6.5 Auswirkung auf die Wissenschaft

- › Verwendung von R bietet am FDZ Bund einen deutlichen **Wettbewerbsvorteil in der emp. Forschung**. Lediglich R ist (am GWAP und via KDFV) am FDZ Bund in der Lage große Datenmengen innerhalb von Sekunden zu analysieren und Datenmengen überhalb des verfügbaren Arbeitsspeichers, wie das BTP, ohne vorherige Variablenselektion auszuwerten.
  - › Vollmaterial kann statt Stichproben angeboten werden. Erleichtert die FDZ Betreuung und GWAP/KDFV-Verfügbarkeit
  - › mehr explorative oder komplexe Analysen sind in kürzerer Zeit möglich.
- › Dazu ist R als frei verfügbare Open-Source Software leicht zu lernen und {arrow} bietet hoch-performante Analysen selbst für R-Basiskenntnisse. Empfehlung an die emp. Wissenschaft, sich verstärkt mit R zu befassen.
- › Für analoge über-RAM-Verarbeitungen bietet Stata aktuell keine Lösung. SAS bietet bei weitem nicht hier gemessene Performanz von R. R hervorragende Möglichkeiten bietet Performanz und Anwenderfreundlichkeit zu übertreffen.
- › Weiteres Vorgehen: **NeSt-Pilotprojekt**. Aus dem realen Use Case Optimierung der Systemkonfiguration und Beratungsbedarfe identifizieren.

## 6.6 Schlussbemerkung

Die Analyse großer Datensätze in der amtlichen Statistik und den Forschungsdatenzen können von einem signifikanten Performanz-Boost profitieren, wenn die vorgestellten Werkzeuge adaptiert werden. **Arrow, DuckDB, Polars und Parquet** – bieten das Potenzial, die Effizienz erheblich zu steigern und neue Analysemöglichkeiten zu eröffnen.

Der Übergang erfordert jedoch eine **koordinierte Anstrengung** von Infrastrukturbetreibern, amtlicher Statistik, Forschungsdatenzentren und Forschenden. Die systematische Wissensaufbau, Dokumentation und Verbreitung von Best Practices wird entscheidend für den Erfolg sein.

Die amtliche Statistik kann von diesen Entwicklungen in

mehrfacher Hinsicht profitieren: **schnellere Erkenntnisgewinnung, umfassendere Analysen** und **effizientere Ressourcennutzung**. Dies trägt letztlich zur Stärkung der evidenzbasierten Politik- und Gesellschaftsberatung bei.

## Anhang

---

Inhalte von Opencode.

## 7

---

Weitere Performancemessungen für kleinere Datenmengen.

---

Tabelle 7

Benchmarkergebnisse für Fallzahlen (kleine Stichproben des BTP)

|        |              |         | [t]                            |          |          |          |
|--------|--------------|---------|--------------------------------|----------|----------|----------|
| 10.000 | {polars}     | Parquet | -                              | 3,3 MB   | 0,4 Sek  | 492,6 MB |
| 10.000 | {data.table} | CSV     | Variablenauswahl               | 4,4 MB   | 0,7 Sek  | 1,3 GB   |
| 10.000 | {arrow}      | Parquet | Variablenauswahl               | 230,7 MB | 1,1 Sek  | 492,6 MB |
| 10.000 | {readr}      | CSV     | Variablenauswahl               | 9,3 MB   | 1,2 Sek  | 1,3 GB   |
| 10.000 | {arrow}      | Parquet | -                              | 229,8 MB | 1,6 Sek  | 492,6 MB |
| 10.000 | {duckdb}     | Parquet | -                              | 4,7 MB   | 2,9 Sek  | 492,6 MB |
| 10.000 | {data.table} | CSV     | -                              | 3,4 GB   | 3,6 Sek  | 1,3 GB   |
| 10.000 | {arrow}      | Parquet | Datenaufteilung (Berichtsjahr) | 229,9 MB | 4,2 Sek  | 486,9 MB |
| 10.000 | {arrow}      | Parquet | -                              | 230,4 MB | 4,6 Sek  | 492,6 MB |
| 10.000 | {data.table} | CSV     | Datenaufteilung (Berichtsjahr) | 5,1 GB   | 4,8 Sek  | 1,0 GB   |
| 10.000 | {arrow}      | Parquet | Datenaufteilung (gleichmäßig)  | 229,9 MB | 7,1 Sek  | 483,5 MB |
| 10.000 | {readr}      | CSV     | -                              | 1,7 GB   | 8,6 Sek  | 1,3 GB   |
| 10.000 | {arrow}      | CSV     | -                              | 27,4 MB  | 15,7 Sek | 1,3 GB   |
| 10.000 | {data.table} | CSV.GZ  | -                              | 4,7 GB   | 21,2 Sek | 421,9 MB |
| 10.000 | {base}       | CSV     | -                              | 9,7 GB   | 4,9 Min  | 1,3 GB   |
| 1.000  | {data.table} | CSV     | Variablenauswahl               | 0,6 MB   | 0,1 Sek  | 129,8 MB |
| 1.000  | {readr}      | CSV     | Variablenauswahl               | 2,7 MB   | 0,2 Sek  | 129,8 MB |
| 1.000  | {polars}     | Parquet | -                              | 3,3 MB   | 0,4 Sek  | 48,1 MB  |
| 1.000  | {data.table} | CSV     | -                              | 341,8 MB | 0,9 Sek  | 129,8 MB |
| 1.000  | {duckdb}     | Parquet | -                              | 4,7 MB   | 1,1 Sek  | 48,1 MB  |
| 1.000  | {arrow}      | Parquet | Variablenauswahl               | 226,2 MB | 1,2 Sek  | 48,1 MB  |
| 1.000  | {arrow}      | Parquet | -                              | 225,3 MB | 1,4 Sek  | 48,1 MB  |
| 1.000  | {arrow}      | CSV     | -                              | 3,5 MB   | 1,5 Sek  | 129,8 MB |
| 1.000  | {data.table} | CSV.GZ  | -                              | 491,7 MB | 2,7 Sek  | 42,1 MB  |
| 1.000  | {data.table} | CSV     | Datenaufteilung (Berichtsjahr) | 510,9 MB | 2,8 Sek  | 104,5 MB |
| 1.000  | {arrow}      | Parquet | Datenaufteilung (Berichtsjahr) | 229,9 MB | 2,9 Sek  | 53,5 MB  |
| 1.000  | {arrow}      | Parquet | -                              | 230,4 MB | 3,7 Sek  | 48,1 MB  |
| 1.000  | {arrow}      | Parquet | Datenaufteilung (gleichmäßig)  | 229,7 MB | 4,0 Sek  | 50,5 MB  |
| 1.000  | {readr}      | CSV     | -                              | 197,4 MB | 4,9 Sek  | 129,8 MB |
| 1.000  | {base}       | CSV     | -                              | 713,4 MB | 23,3 Sek | 129,8 MB |

Tabelle 8

Benchmarkergebnisse für Regression (kleine Stichproben des BTP)

|        |         |         | [t]                            |          |          |          |
|--------|---------|---------|--------------------------------|----------|----------|----------|
| 10.000 | {dplyr} | CSV     | Variablenauswahl               | 17,6 MB  | 1,3 Sek  | 1,3 GB   |
| 10.000 | {arrow} | Parquet | Datenaufteilung (Berichtsjahr) | 459,3 MB | 5,0 Sek  | 486,9 MB |
| 10.000 | {dplyr} | CSV     | -                              | 1,8 GB   | 10,8 Sek | 1,3 GB   |
| 1.000  | {dplyr} | CSV     | Variablenauswahl               | 4,2 MB   | 0,3 Sek  | 129,8 MB |
| 1.000  | {dplyr} | CSV     | -                              | 206,9 MB | 6,4 Sek  | 129,8 MB |
| 1.000  | {arrow} | Parquet | Datenaufteilung (Berichtsjahr) | 452,5 MB | 6,8 Sek  | 53,5 MB  |

## Literatur

---

Gomolka, Matthias, Jannick Blaschke und Christian Hirsch (2021). *Working with Large Data at the RDSC*. Technical Report 2021-04. Frankfurt am Main: Deutsche Bundesbank. URL: <https://www.bundesbank.de/en/bundesbank/research/rdsc/publications/methodology-reports/working-with-large-data-at-the-rdsc-623988>.